

COMPUTER NETWORKS

Comprehensive Semester Examination Master Question Bank

Program: Bachelor of Technology (B.Tech) • Computer Science & Engineering (CSE)

Core Focus: High-Yield, Concise Exam Blueprints & Solutions (No Tables version for Core Questions)

PART A: CORE CONCEPTS & ALGORITHMS

Q1. Explain the hidden terminal problem in Wireless LANs and how CSMA/CA solves it. (5 Marks)

1. The Problem

The Hidden Terminal Problem occurs in wireless environments when two stations (A and C) are out of radio range of each other but can both communicate with a central base station B. If A transmits to B, C cannot detect the carrier signal, assumes the channel is free, and also begins transmitting. This causes a destructive collision at station B, rendering both frames unreadable.

2. The Solution: CSMA/CA with RTS/CTS

CSMA/CA solves this problem by using explicit Request-to-Send (RTS) and Clear-to-Send (CTS) control frames to channel-reserve the medium:

- Station A sends an **RTS frame** to B, specifying the duration needed for data transmission.
- Station B broadcasts a **CTS frame** to all nodes in its range. This frame contains the time reservation.
- Station C hears B's broadcasted CTS, learns the channel is locked, and defers its own transmission, preventing collision.

Quick Blueprint: $A \rightarrow B \leftarrow C$ (Collision at B) | Solved by: $A [RTS] \rightarrow B [CTS \text{ Broadcast}] \rightarrow \text{Silences } C \rightarrow \text{Safe data transfer from } A$.

Q2. Explain Selective Repeat ARQ and compare it with Go-Back-N ARQ. (5 Marks)

1. Selective Repeat ARQ

Selective Repeat (SR) ARQ is an error-control sliding window protocol where both the sender and receiver maintain a window of equal size W . The receiver accepts and buffers out-of-order frames. When a frame drop or corruption occurs, the receiver issues a Negative Acknowledgment (NAK) or waits, and the sender retransmits **only** that individual missing frame, minimizing bandwidth retransmission waste.

2. Structural Comparison Table

Feature	Go-Back-N (GBN) ARQ	Selective Repeat (SR) ARQ
Receiver Window Size	Fixed at 1 (Drops out-of-order frames)	Size W (Buffers out-of-order frames)
Retransmission Scope	The failed frame AND all subsequent frames sent	Only the single failed frame
Efficiency Factor	Low under high channel error states	High; avoids redundant retransmissions
Memory Cost	Low; no receiver buffer space required	High; requires sorting buffer pools

Quick Scenario Blueprint: If frames 0, 1, 2, 3 are sent and frame 1 is lost:

- Go-Back-N: Drops 2 and 3; retransmits 1, 2, and 3.
- Selective Repeat: Buffers 2 and 3; retransmits *only* 1.

Q3. Given the IP address 200.10.20.0/24, create four equal-sized subnets and write the usable IP ranges. (5 Marks)

1. Mathematical Subnetting Setup

The base address is Class C with a 24-bit network prefix, leaving 8 bits for hosts ($2^8 = 256$ total addresses). To create 4 subnets ($4 = 2^2$), we must borrow 2 bits from the host bits pool. New Subnet Mask: $24 + 2 = \mathbf{26}$ (Binary: 255.255.255.192). Block Size per Subnet: $256 / 4 = 64$ total addresses.

2. Subnet Allocation & Usable Ranges

- **Subnet 1:** ID: 200.10.20.0/26 | Usable Range: 200.10.20.1 to 200.10.20.62 | Broadcast: 200.10.20.63
- **Subnet 2:** ID: 200.10.20.64/26 | Usable Range: 200.10.20.65 to 200.10.20.126 | Broadcast: 200.10.20.127
- **Subnet 3:** ID: 200.10.20.128/26 | Usable Range: 200.10.20.129 to 200.10.20.190 | Broadcast: 200.10.20.191
- **Subnet 4:** ID: 200.10.20.192/26 | Usable Range: 200.10.20.193 to 200.10.20.254 | Broadcast: 200.10.20.255

Note: In each subnet, the first address (Subnet ID) and last address (Broadcast) are unassignable, yielding 62 usable host IPs per block.

Q4. Compare IPv4 and IPv6 header structures. (5 Marks)

1. Core Design Modifications

IPv4 features a variable-length header (20 to 60 bytes) with 14 fields, creating processing overhead at intermediate routers. IPv6 features a simplified fixed-length base header of 40 bytes with only 8 fields, speeding up hardware routing processing by offloading complex operations to optional Extension Headers.

2. Primary Attribute Comparison

Feature	IPv4 Header	IPv6 Header
Address Length	32 bits (4 Bytes)	128 bits (16 Bytes)
Header Size	Variable (Minimum 20 Bytes)	Fixed (Strictly 40 Bytes)
Field Count	14 fields	8 fields
Header Checksum	Present; updated at every single hop router	Removed entirely (handled at Layers 2/4)
Fragmentation	Supported at intermediate routers	Executed exclusively by source host
TTL Field	Time to Live (TTL)	Renamed to Hop Limit

Q5. Differentiate between TCP and UDP with suitable examples. (5 Marks)

1. Definitions

TCP is a reliable, connection-oriented protocol that ensures sequential end-to-end data delivery via acknowledgments and flow checks. UDP is an ultra-lightweight, connectionless, best-effort protocol that pushes segments immediately without arrival guarantees or state tracking.

2. Field Matrix & Applications

Metric	TCP	UDP
Connection Style	Connection-oriented (3-way handshake)	Connectionless (Direct transmission)
Reliability Target	Guaranteed; handles retransmissions	Unreliable; best-effort delivery
Byte Ordering	Strict sequential arrangement	No ordering tracking logic
Header Size	Variable (20 to 60 Bytes)	Fixed at 8 Bytes
Applications	Web Traffic (HTTP), Mail (SMTP), Files (FTP)	Streaming Video, Voice (VoIP), DNS, Gaming

Q6. Explain the Token Bucket algorithm. How is it different from the Leaky Bucket algorithm? (5 Marks)

1. Token Bucket Algorithm

Token Bucket is a traffic shaping algorithm that permits bursty data transmission. A virtual bucket holds up to B tokens, which accumulate at a constant rate of R tokens per second. When an application submits a packet, it must extract tokens from the bucket to transmit. If the bucket is full, the application can send a sudden burst of data up to size B instantly into the network.

2. Structural Divergences

- **Leaky Bucket:** Collects bursty data traffic and shapes it into a strictly constant, rigid output stream. Wastes idle bandwidth as it cannot store credits. Primary tool for rigid traffic policing.
- **Token Bucket:** Stores transmission credits (tokens) when the network is idle. This allows safe, controlled bursts when traffic resumes, making it ideal for variable, bursty data applications.

Analogy Blueprint: Leaky Bucket functions like a funnel that drips water at one steady rate. Token Bucket functions like a credit account where saved tokens allow large burst transactions.

Q7. Explain recursive DNS resolution when a user types a URL in the browser. (5 Marks)

1. Definition

When a browser requests a domain translation, recursive resolution places the full resolution workload onto a single server—the Local DNS Resolver. The local resolver iteratively queries the global nameserver hierarchy until it finds the final IP address mapping.

2. Operational Resolution Pipeline

1. **Local Query:** The browser issues a recursive query to the Local DNS Resolver (ISP).
2. **Root Hop:** The Local Resolver queries a global Root Nameserver (.), which directs it to the appropriate Top-Level Domain (TLD) Server.
3. **TLD Hop:** The resolver queries the TLD Server (e.g., .com), which provides the IP address of the domain's Authoritative Nameserver.
4. **Authoritative Hop:** The resolver queries the Authoritative Nameserver, which holds the primary records and returns the final destination IP.
5. **Client Return:** The resolver caches the record and returns the IP to the browser to launch the connection.

Q8. Explain the working principles of DNS, HTTP, and FTP protocols. (5 Marks)

1. DNS (Domain Name System)

Operates over UDP Port 53. It functions as a distributed hierarchical directory database to convert alphanumeric web addresses (e.g., google.com) into logical IP addresses readable by routing hardware.

2. HTTP (Hypertext Transfer Protocol)

Operates over TCP Port 80 (HTTPS uses 443) on a stateless Request-Response model. A client browser issues request verbs (e.g., `GET /home`), and the target server replies with status headers (e.g., `200 OK`) and the requested payload, tracking no user state between calls.

3. FTP (File Transfer Protocol)

A stateful file transfer protocol using two separate TCP connections to achieve out-of-band control signaling: a *Control Connection* (Port 21) for commands and authentication, and a *Data Connection* (Port 20) that opens and closes dynamically to transfer files.

Q9. What is BGP? Explain its role as an Inter-domain routing protocol. (5 Marks)

1. Definition

BGP (Border Gateway Protocol) is the core path-vector routing protocol that connects the global internet. It operates at the application layer over TCP port 179 and is classified as an External Gateway Protocol (EGP).

2. Role in Inter-Domain Routing

The global internet is divided into independent networks called Autonomous Systems (AS). BGP routes packets between these Autonomous Systems:

- **Path-Vector Routing:** BGP updates include a complete list of AS numbers traversed to reach a destination network, allowing routers to spot and drop routing loops instantly.
- **Policy-Driven Path Selection:** BGP does not select paths based on simple link metrics or hop counts. Instead, path selection is driven by complex enterprise policies, business contracts, and security rules.

Q10. Explain ARP protocol with suitable example. (5 Marks)

1. Core Concept

ARP (Address Resolution Protocol) dynamically maps a known Network Layer logical IP address to a corresponding physical MAC address within a local area network link.

2. Operational Example

Suppose Host A (IP: `192.168.1.5`) wants to communicate with Host B (IP: `192.168.1.20`) on the same local network switch but lacks its MAC coordinates:

- Host A broadcasts an **ARP Request** frame across the network segment: *"Who owns 192.168.1.20? Tell 192.168.1.5."* Destination MAC is set to broadcast `FF:FF:FF:FF:FF:FF`.
- All nodes read the frame, but only Host B recognizes its own IP. It sends a unicast **ARP Reply** frame directly back to Host A: *"I own that IP; my MAC is 00:1A:2B:3C:4D:5E."*
- Host A saves the entry in its local ARP cache and wraps its data frame to transmit it directly to Host B.

Q11. Explain CSMA/CD protocol briefly. (5 Marks)

1. Core Concept

CSMA/CD (Carrier Sense Multiple Access with Collision Detection) is a media access control protocol used in shared, half-duplex wired Ethernet networks to coordinate transmission media access and handle data collisions.

2. Execution Steps

1. **Carrier Sensing:** A station monitors the physical wire. If it senses the line is idle, it begins transmitting its frame.
2. **Collision Detection:** While sending, the station monitors line voltage levels. A voltage spike indicates a collision occurred.
3. **Jam Signal:** The station immediately stops data transmission and sends a brief Jam Signal to ensure all other nodes discard the partial frame.
4. **Backoff:** The station runs the Exponential Backoff algorithm to calculate a random wait time before checking the line to retry.

Q12. Explain Pure ALOHA and Slotted ALOHA. (5 Marks)

1. Pure ALOHA

A completely uncoordinated random access protocol. Whenever a station has a data frame ready, it transmits immediately. This lack of coordination leads to frequent collisions. The vulnerable collision window is $2 \times T_{fr}$, limiting maximum channel throughput efficiency to **18.4%**.

2. Slotted ALOHA

Introduces time synchronization. The channel timeline is divided into strict slots equal to one frame's duration (T_{fr}). Stations must buffer data and wait to transmit until the exact start of the next slot. This cuts the vulnerable collision window in half to $1 \times T_{fr}$ and doubles maximum throughput efficiency to **36.8%**.

Q13. Explain Hamming Code and its application. (5 Marks)

1. Mathematical Concept

Hamming Code is a Forward Error Correction (FEC) block code that detects and automatically corrects single-bit errors inside a received frame without requesting a retransmission. It inserts r parity check bits at positions that are powers of 2 ($1, 2, 4, 8 \dots$) within an m -bit data word, satisfying the constraint: $2^r \geq m + r + 1$. The receiver evaluates these parity checks to generate a binary pointer (syndrome) showing the exact location of the flipped bit.

2. Real-World Applications

- **ECC Server Memory:** Used in enterprise servers and data centers to automatically detect and correct single-bit memory corruptions caused by cosmic rays or electrical noise.
- **Storage Architectures:** Integrated into storage arrays to protect data blocks during read/write cycles.

Q14. Differentiate between Bridge, Switch, Router, and Gateway. (5 Marks)

1. Structural Overview

These hardware components link separate parts of a network topology, operating at different layers of the OSI model:

- **Bridge (Layer 2):** Connects two local network segments. It filters traffic by reading MAC addresses to reduce collisions.
- **Switch (Layer 2):** An intelligent, multi-port bridge. It maintains an internal MAC address table to forward incoming frames directly to the target destination port rather than broadcasting them.
- **Router (Layer 3):** Links entirely separate networks (e.g., bridging a private LAN to the public Internet). It routes data packets using logical IP addresses and routing tables.
- **Gateway (Layers 4-7):** A complex protocol converter that translates data payloads across completely different network architectures and protocol stacks.

Q15. Explain NAT (Network Address Translation). (5 Marks)

1. Core Purpose

NAT is an internet engineering method designed to delay the depletion of public IPv4 addresses. It allows an entire local network using unroutable private IPs (e.g., `192.168.1.x`) to share a single public IP address to communicate with the outside internet.

2. Operational Workflow (NAPT)

When an internal host sends an outbound packet, the NAT edge router replaces the private source IP and port with its own public IP and a unique port number, logging the mapping into an internal table. When return traffic arrives, the router checks the table, translates the public coordinates back into the original private IP/port mapping, and forwards the packet onto the local switch.

Core Advantages: Preserves public IPv4 addresses and enhances security by masking internal network structures from direct external access.

PART B: ADVANCED NETWORK ROUTING & FLOW CONTROLS

Q1. Explain the OSI Reference Model with functions of each layer. (5 Marks)

1. Seven-Layer Architecture

The Open Systems Interconnection (OSI) model standardizes network communication into seven independent, layered abstraction tiers. Data wraps downward during encapsulation at the sender and unwraps upward during decapsulation at the receiver:

- **7. Application:** Interface providing direct network services to software applications (HTTP, FTP, SMTP).
- **6. Presentation:** Formats, encrypts, decompresses, and translates data syntaxes.

- **5. Session:** Establishes, manages, checkpoints, and terminates active communication dialogues.
- **4. Transport:** Ensures reliable end-to-end delivery, segmentation, and flow checking (TCP, UDP).
- **3. Network:** Handles logical IP addressing and routes data packets across networks.
- **2. Data Link:** Packages packets into frames, handles hardware MAC addressing, and performs error checks.
- **1. Physical:** Moves raw, unstructured bit streams over physical hardware media using electrical or optical signaling.

Q2. Differentiate between Connection-Oriented and Connectionless services. (5 Marks)

1. Core Concepts

A Connection-Oriented service requires a dedicated logical path to be established between endpoints via a handshake before data can flow. A Connectionless service transmits each packet independently with its full destination address, requiring no connection setup.

2. Key Differences

Metric	Connection-Oriented Service	Connectionless Service
Handshake Phase	Mandatory connection setup and teardown	None; transmits immediately
Path Consistency	All packets follow the exact same path	Packets route dynamically over different paths
Delivery Sequence	Guaranteed in-order delivery	Packets can arrive out of sequence
Reliability	High; uses tracking and retransmissions	Best-effort delivery; no built-in tracking
Examples	TCP, Frame Relay	UDP, IP, ICMP

Q3. Explain Dijkstra's Algorithm with a suitable example. (5 Marks)

1. Conceptual Purpose

Dijkstra's Algorithm is a link-state routing algorithm that calculates the shortest path from a single source node to all other nodes in a weighted graph network by maintaining a running list of lowest-cost paths to unvisited nodes.

2. Numerical Graph Verification Example

Consider 4 nodes (A, B, C, D) with link weights: $A \rightarrow B = 1$, $A \rightarrow C = 4$, $B \rightarrow C = 2$, $B \rightarrow D = 6$, $C \rightarrow D = 3$. Let A be the source node:

- **Init:** Distance $D(A) = 0$; all other node distances are set to infinity: $D(B) = \infty$, $D(C) = \infty$, $D(D) = \infty$. Unvisited set = $\{A, B, C, D\}$.
- **Evaluate A:** Closest node is A ($D(A)=0$). Check neighbors: $D(B)$ drops from $\infty \rightarrow 1$. $D(C)$ drops from $\infty \rightarrow 4$. Mark A visited. Remaining unvisited = $\{B, C, D\}$.
- **Evaluate B:** Closest unvisited node is B ($D(B)=1$). Check unvisited neighbors: path to C via B is $1 + 2 = 3$. Since $3 < 4$, update $D(C) = 3$. Path to D via B is $1 + 6 = 7$. Update $D(D) = 7$. Mark B visited. Remaining unvisited = $\{C, D\}$.
- **Evaluate C:** Closest unvisited node is C ($D(C)=3$). Check neighbor D: path to D via C is $3 + 3 = 6$. Since $6 < 7$, update $D(D) = 6$. Mark C visited.
- **Evaluate D:** Only D remains ($D(D)=6$). The algorithm terminates.

Final Solution Pointers: Shortest distance from source A: to B = 1 | to C = 3 | to D = 6.

Q4. Explain Link State Routing process. (5 Marks)

1. Definition

Link State Routing (LSR) is a dynamic routing architecture where every router independently constructs a complete, identical map of the entire network topology to calculate its own optimal path choices.

2. Five-Step Operational Blueprint

1. **Neighbor Discovery:** Routers periodically send broadcast *Hello packets* to discover directly connected neighbors.
2. **Cost Measurement:** The router tests echo delay or bandwidth to measure the cost metric for each direct link.
3. **LSP Creation:** The router builds a Link State Packet (LSP) containing its ID, sequence number, and direct link costs.
4. **Flooding:** The router floods its LSP across the network, ensuring all routers synchronize an identical Link State Database (LSDB).
5. **Dijkstra Calculation:** Each router independently runs Dijkstra's algorithm on the LSDB map to build its local routing tables.

Q5. Explain Distance Vector Routing with suitable example. (5 Marks)

1. Mechanism

In Distance Vector Routing (DVR), routers do not know the complete network map. Instead, each router maintains a table listing its known *Distance* (total cost metric) and *Vector* (next-hop destination) to every network. Routers periodically share their entire routing tables only with their directly connected neighbors.

2. Example & Loop Vulnerability

In a linear network **A — (cost 2) — B — (cost 3) — C**, Node A initially does not know about Node C ($D(C) = \infty$). When neighbor B shares its table, which lists its cost to C as 3, Node A calculates its total cost to C: $2 + 3 = 5$. Since $5 < \infty$, A updates its entry for C to Cost 5 via Next-Hop B.

The Count-to-Infinity Problem: If link B-C breaks, B sets its cost to C to infinity. However, if A sends an outdated update first, B may mistakenly think A has an alternate path to C. The nodes will increment costs endlessly ($16 \rightarrow 17 \rightarrow 18 \dots$), creating a routing loop.

Q6. Explain Hierarchical Routing and its advantages. (5 Marks)

1. Core Concept

Hierarchical Routing splits large networks into layered geographic areas or clusters (e.g., OSPF areas). Routers within a region only maintain detailed routing paths for internal local nodes. They do not track the internal structure of external regions, directing all external traffic to a single high-level gateway or backbone router pointer instead.

2. Key Advantages

- **Table Contraction:** Drastically shrinks routing table size. Instead of tracking thousands of individual external paths, a router only tracks local nodes and external area gateways, saving memory.
- **Bandwidth Preservation:** Restricts routing updates to local regions, preventing massive flooding across the wider network.
- **Fault Isolation:** Topology changes or link failures inside one region are managed locally and do not trigger recomputations in distant regions.

Q7. Discuss the Sliding Window Protocol mechanism for flow control. (5 Marks)

1. Functional Objective

The Sliding Window Protocol is a flow control mechanism that prevents a fast sender from overwhelming a slower receiver with too much data. It allows the sender to transmit multiple data frames continuously without waiting for an individual acknowledgment (ACK) for each frame.

2. Operational Mechanics

Both sender and receiver maintain a buffer window of allowed sequence numbers. The sender's window size W defines the maximum number of unacknowledged frames it can transmit. As the sender transmits frames, it fills open slots in its window. When the receiver successfully processes a frame, it returns an ACK. Upon receiving the ACK, the sender shifts ("slides") its window boundary forward, freeing up slots to transmit new data frames.

Q8. Compare 1-bit Sliding Window and Go-Back-N ARQ. (5 Marks)

1. Structural Definitions

A 1-bit Sliding Window protocol is a simple stop-and-wait mechanism where the sender window size is strictly fixed at $W = 1$. The sender must wait for an individual ACK after every single frame. Go-Back-N (GBN) ARQ is a pipelined protocol where the sender window can expand up to $W = 2^m - 1$, allowing multiple frames to be sent continuously before receiving an ACK.

2. Feature Matrix

Metric Parameter	1-bit Sliding Window	Go-Back-N (GBN) ARQ
Sender Window Size	Strictly limited to 1	Expanded range up to $2^m - 1$
Pipelining Capability	No; cannot send data during transit delays	Yes; fills link pipe with continuous data
Error Handling	Retransmits only the single timed-out frame	Retransmits the failed frame AND all subsequent frames sent
Channel Efficiency	Extremely low on long-delay links	High; maximizes link capacity utilization

Q9. Discuss the CSMA/CD protocol and collision handling in Ethernet. (5 Marks)

1. CSMA/CD Operation

CSMA/CD manages media access on shared half-duplex wired Ethernet networks. Devices listen to the wire before transmitting to avoid collisions and monitor line voltage during transmission to catch any overlapping signals.

2. Collision Handling Pipeline

- **Transmission Halt:** The instant a node's transceiver detects a collision (via a line voltage spike), it immediately stops data transmission.
- **Jam Signal:** The station broadcasts a 32-to-48-bit Jam Signal to ensure all other devices on the segment recognize the collision and discard the partial frame.
- **Exponential Backoff:** Nodes increment their collision counter (k) and run the Binary Exponential Backoff algorithm, selecting a random slot multiplier r from the range $0 \leq r < 2^k$. The node waits for $r \times 51.2 \mu\text{s}$ slot times before attempting to retransmit.

COMPUTER NETWORKS

Semester Exam Preparation Question Bank (Q10 - Q23)

Course: Bachelor of Technology (B.Tech) in Computer Science & Engineering
Format: High-Yield Concise Reference Guide

Q10. Explain Ethernet (IEEE 802.3) frame structure. (5 Marks)

1. Introduction

The IEEE 802.3 Standard (Ethernet) organizes data at the Data Link layer into a standardized format called an Ethernet Frame. The size of a standard frame ranges from a minimum of 64 bytes to a maximum of 1518 bytes (excluding the Preamble and SFD).

2. Frame Fields and Functions

- **Preamble (7 Bytes):** Alternating 0s and 1s used by the receiver to synchronize its clock circuitry with incoming electrical signals.
- **Start Frame Delimiter / SFD (1 Byte):** The bit pattern `10101011`. The final two bits signal that synchronization is complete and data follows immediately.
- **Destination Address (6 Bytes):** Contains the physical MAC address of the receiving node.
- **Source Address (6 Bytes):** Contains the physical MAC address of the transmitting node.
- **Length / Type (2 Bytes):** If value is ≤ 1500 , it indicates the Length of the data field. If it is > 1500 , it defines the protocol Type of the Network Layer (e.g., IPv4).
- **Data / Payload (46 to 1500 Bytes):** Contains the network layer packet. If the payload is under 46 bytes, dummy bits called Padding are appended to hit the mandatory 64-byte minimum frame size.
- **Frame Check Sequence / FCS (4 Bytes):** Contains a 32-bit Cyclic Redundancy Check (CRC) error-detection code. Corrupt frames are dropped.

Quick Size Blueprint:

[7B Preamble] → [1B SFD] → [6B Dest MAC] → [6B Src MAC] → [2B Type] → [46-1500B Data] → [4B FCS]

- *Minimum Frame Size: $6 + 6 + 2 + 46 + 4 = 64$ Bytes (Excluding Preamble/SFD)*
- *Maximum Frame Size: $6 + 6 + 2 + 1500 + 4 = 1518$ Bytes (Excluding Preamble/SFD)*

Q11. Explain TCP Congestion Control mechanisms. (5 Marks)

1. Introduction

TCP uses network congestion control to prevent a sender from overwhelming the network infrastructure. It dynamically adjusts the *cwnd* (Congestion Window)—the amount of data sent before receiving an ACK—using a threshold state variable called *ssthresh* (Slow Start Threshold).

2. Core Mechanisms

- **a. Slow Start:** *cwnd* starts small (e.g., 1 MSS). For every individual ACK received, *cwnd* increases by 1 MSS. This results in exponential growth ($1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \dots$) per Round Trip Time (RTT). It terminates when $cwnd \geq ssthresh$ or a packet drop occurs.
- **b. Congestion Avoidance:** Triggered when $cwnd \geq ssthresh$. To probe network capacity safely, *cwnd* increases linearly by 1 MSS only after an entire window of ACKs is received (+1 MSS per RTT).
- **c. Fast Recovery:** Triggered by 3 duplicate ACKs (mild congestion). Instead of dropping *cwnd* to 1, TCP performs a Fast Retransmit of the missing segment and optimizes the window:

$$ssthresh = cwnd / 2$$

$$cwnd = ssthresh + 3 \text{ MSS}$$

It skips Slow Start entirely and directly enters Congestion Avoidance.

Quick Reaction Blueprint:

- *Timeout (Severe Congestion):* $ssthresh = cwnd / 2$, reset $cwnd = 1 \text{ MSS}$, restart via Slow Start.
- *3 Duplicate ACKs (Mild Congestion):* $ssthresh = cwnd / 2$, drop $cwnd$ to new $ssthresh$, skip to Congestion Avoidance.

Q12. Explain QoS and techniques to improve QoS in networks. (5 Marks)

1. What is QoS?

QoS (Quality of Service) is a collection of technologies used to manage network resources and guarantee performance levels for data traffic. It ensures critical, time-sensitive applications (like VoIP or video calls) receive priority over less critical traffic (like emails).

Core performance metrics include: **Delay (Latency)** (time taken for a packet to travel from source to destination), **Jitter** (variation in packet arrival times), and **Packet Loss** (percentage of packets dropped during congestion).

2. Techniques to Improve QoS

- **A. Scheduling (Queue Management):** Routers sort packets using internal queues. *Priority Queuing (PQ)* empties high-priority queues completely before lower ones are processed. *Weighted Fair Queuing (WFQ)* allocates bandwidth dynamically by assigning weights, preventing traffic starvation.
- **B. Traffic Shaping and Policing:** Regulates data rates to smooth out bursts. *Leaky Bucket* collects data and releases it at a strict, constant rate, dropping overflows. *Token Bucket* generates tokens at a constant rate and allows bursts as long as tokens are available.

- **C. Resource Reservation (RSVP):** A signaling protocol where a sending host reserves a specific amount of bandwidth across routers along the path before transmission begins.

Quick Blueprint:

- *For Real-Time Video/Voice: Prioritize packets and reduce Jitter using Priority Queuing.*
- *For Sudden Data Spikes: Smooth out bursty traffic using the Token Bucket shaper.*

Q13. Explain Email system architecture using SMTP, POP3, and IMAP. (5 Marks)

1. Email Architecture Overview

The email framework operates on a client-server model split into Mail Submission (pushing) and Mail Retrieval (pulling). It uses three components: **User Agent (UA)** (the application interface like Outlook/Gmail), **Message Transfer Agent (MTA)** (routing servers), and **Mail Delivery Agent (MDA)** (deposits mail into mailboxes).

2. Core Protocols

- **SMTP (Simple Mail Transfer Protocol):** A push protocol running over TCP port 25 or 587. It transfers mail from sender UA to server, and between intermediate servers.
- **POP3 (Post Office Protocol v3):** A pull protocol over TCP port 110 or 995. It operates on a "download-and-delete" model, transferring mail to a local client and erasing it from the server, preventing multi-device sync.
- **IMAP (Internet Message Access Protocol):** An advanced pull protocol over TCP port 143 or 993. It stores messages permanently on the remote server and synchronizes organizational changes across all user devices in real-time.

Quick Summary:

- *SMTP: Push protocol | Transient storage | No sync*
- *POP3: Pull protocol | Local device storage | No multi-device sync*
- *IMAP: Pull protocol | Central server storage | Real-time multi-device sync*

Q14. Differentiate between Packet Filtering Firewall and Application Gateway Firewall. (5 Marks)

1. Conceptual Overview

A **Packet Filtering Firewall** is a low-level network filter that inspects individual packet headers (IP addresses and ports) at the network/transport layers without reading the data payload. An **Application Gateway Firewall** (Proxy Firewall) is a high-level security layer acting as an intermediate proxy that inspects the complete application layer payload to enforce content policies.

2. Core Architectural Differences

- **Operating Layer:** Packet Filtering runs at Layers 3 & 4 (Network/Transport). Application Gateway operates at Layer 7 (Application).

- **Inspection Depth:** Packet Filtering checks headers only. Application Gateway performs deep packet inspection on the actual content/data payload.
- **Connection Style:** Packet Filtering allows a direct link between source and destination. Application Gateway separates connections, acting as a middleman proxy.
- **Performance & Resource Cost:** Packet Filtering is very fast with low overhead. Application Gateway is slower and resource-heavy due to processing depth.
- **Security Tier:** Packet Filtering is vulnerable to spoofing and application-level bypasses. Application Gateway provides high isolation and robust defense.

Quick Blueprint:

- Packet Filter checks identity cards/badges (IP/Port) at the door.
- Application Gateway opens up the suitcase to inspect inside contents (Payload).

Q15. Discuss the RSA Public Key Encryption algorithm with a suitable numerical example. (5 Marks)

1. Introduction & Principle

RSA is an asymmetric cryptography algorithm utilizing a Public Key for encryption and a distinct Private Key for decryption. Its underlying security relies on the mathematical difficulty of factoring large prime numbers.

2. Mathematical Steps

1. Choose distinct primes p and q .
2. Compute Modulus: $n = p \times q$.
3. Compute Totient: $\phi(n) = (p - 1)(q - 1)$.
4. Choose Public Exponent e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.
5. Calculate Private Exponent d : $(d \times e) \equiv 1 \pmod{\phi(n)}$.

• Public Key = $\{e, n\}$ | Private Key = $\{d, n\}$

3. Numerical Example

- **Key Generation:** Let $p = 3$, $q = 11$.

$$n = 3 \times 11 = 33$$

$$\phi(n) = (3 - 1)(11 - 1) = 2 \times 10 = 20$$

Let $e = 3$ (valid since $\gcd(3, 20) = 1$).

Find d : $(d \times 3) \equiv 1 \pmod{20} \rightarrow d = 7$ (since $7 \times 3 = 21 \equiv 1 \pmod{20}$).

- **Encryption:** For message Plaintext $M = 7$:

$$C = M^e \pmod{n} = 7^3 \pmod{33} = 343 \pmod{33} = 13.$$

- **Decryption:** Convert Ciphertext $C = 13$ back:

$$M = C^d \pmod{n} = 13^7 \pmod{33} = 7.$$

Formula Blueprint:

- Encryption: $C = M^e \pmod{n}$
- Decryption: $M = C^d \pmod{n}$

Q16. Explain Digital Signatures and Non-Repudiation. (5 Marks)

1. What is a Digital Signature?

A Digital Signature is a mathematical cryptographic mechanism used to validate the authenticity, identity, and integrity of digital data. It binds a sender's unique identity to a message using Public Key Infrastructure (PKI).

2. Working Mechanism

- **Signing (Sender Side):** The sender passes the message through a cryptographic hash function to create a fixed-size *Message Digest*. This digest is encrypted using the sender's secret **Private Key** to generate the Digital Signature, which is appended to the message.
- **Verification (Receiver Side):** The receiver generates a local hash of the received message and decrypts the signature using the sender's public key. If the two match perfectly, it guarantees the message was unaltered and genuinely authored by the owner of the key.

3. Non-Repudiation

Non-Repudiation is a security property ensuring that a sender cannot deny creating or transmitting a specific piece of data. Since the digital signature requires the sender's unique, tightly protected Private Key to generate, a mathematically verifiable signature acts as irrefutable proof of origin to any third-party auditor, preventing any subsequent denial of the transaction.

Core Linkage:

- *Authenticity → Decrypted successfully with sender's Public Key.*
- *Integrity → Local Hash matches decrypted Message Digest.*
- *Non-Repudiation → Only the sender's Private Key could have generated the signature.*

Q17. Define Socket Programming and explain TCP Client–Server Communication. (5 Marks)

1. What is Socket Programming?

Socket Programming is the process of establishing communication endpoints across networked software processes. A **Socket** is a logical endpoint uniquely defined by the binding of an **IP Address** (node identifier) and a **Port Number** (process identifier).

2. TCP Client–Server Workflow

TCP is connection-oriented, demanding explicit system calls to coordinate an active connection path:

- **Server-Side (Passive Endpoint):**
 1. `socket()`: Provisions the network endpoint descriptor.
 2. `bind()`: Hooks the socket to a local address structure (IP and Port).
 3. `listen()`: Transitions socket to passive listening state for incoming client requests.
 4. `accept()`: Blocks execution until a request is intercepted; allocates a new dedicated socket for the stream.
 5. `read()` / `write()`: Handles bi-directional communication.

- **Client-Side (Active Endpoint):**

1. `socket()` : Provisions client endpoint descriptor.
2. `connect()` : Dispatches an active connection request targeting the server's IP/Port (executing the TCP 3-way handshake).
3. `write()` / `read()` : Executes the data exchange.

System Call Pipeline:

- *Server: `socket()` → `bind()` → `listen()` → `accept()` → `read/write` → `close()`*
- *Client: `socket()` → `connect()` → `write/read` → `close()`*

Q18. Write pseudo-code for a UDP Echo Server. (5 Marks)

1. Architectural Concept

A UDP Echo Server is a connectionless application. It bypasses the TCP connection handshake pipeline (no `listen` or `accept` system calls required), binds immediately to a target listening port, and loops indefinitely extracting incoming raw datagrams. For every valid packet processed, it pulls the sender's address structure and bounces the identical data payload right back.

2. UDP Echo Server Pseudo-code

```
CONSTANT PORT = 8080
CONSTANT BUFFER_SIZE = 1024

FUNCTION start_udp_echo_server():
    server_socket = create_socket(AF_INET, SOCK_DGRAM)

    IF server_socket == ERROR:
        RETURN

    server_address = initialize_address(INADDR_ANY, PORT)
    bind_status = bind_socket(server_socket, server_address)

    IF bind_status == ERROR:
        close_socket(server_socket)
        RETURN

    LOOP FOREVER:
        buffer = allocate_memory(BUFFER_SIZE)
        client_address = allocate_memory(sizeof(Address_Structure))

        received_bytes = recv_from(server_socket, buffer, BUFFER_SIZE, client_address)

        IF received_bytes > 0:
            send_to(server_socket, buffer, received_bytes, client_address)

            free_memory(buffer)
            free_memory(client_address)

        close_socket(server_socket)
    END_FUNCTION
```

Execution Sequence:

Socket() [Create UDP] → Bind() [Assign Port] → RecvFrom() [Await Data] → SendTo() [Echo Back]

Q19. Calculate the CRC for the data block 1101011011 using generator polynomial $G(x) = x^4 + x + 1$. (5 Marks)

1. Setup Parameters

- **Data Block:** 1101011011
- **Generator Polynomial:** $G(x) = x^4 + x + 1$ expressed in binary as 10011 (Degree $n = 4$).
- **Appended Dividend:** Append 4 zeros to the frame $\rightarrow$ 11010110110000.

2. Modulo-2 Division (XOR Arithmetic)

```

              1 1 0 0 0 0 1 0 1 0 (Quotient)
1 0 0 1 1 | 1 1 0 1 0 1 1 0 1 1 0 0 0 0
            1 0 0 1 1
            -----
            0 1 0 0 1 1
              1 0 0 1 1
              -----
              0 0 0 0 0 1 1 0 1 1 (Bits pulled down)
                1 0 0 1 1
                -----
                0 1 0 1 0 0 (Final zero pulled down)
                  1 0 0 1 1
                  -----
                  0 0 1 1 1 0 (Calculated Remainder)
```

3. Operational Verification Steps

1. 11010 $\oplus$ 10011 = 01001. Pull down 1 $\rightarrow$ 10011.
2. 10011 $\oplus$ 10011 = 00000. Pull down bits until a leading 1 is formed $\rightarrow$ 11011.
3. 11011 $\oplus$ 10011 = 01000. Pull down 0 $\rightarrow$ 10000.
4. 10000 $\oplus$ 10011 = 00011. Pull down final 0 $\rightarrow$ 1110.

Final Solution:

- CRC Check Bits: 1110
- Transmitted Frame String: 11010110111110

Q20. Explain Error Detection and Error Correction techniques. (5 Marks)

1. Core Concepts & Redundancy

Error Detection: Verifying if structural bits were inverted ($0 \rightarrow 1$ or $1 \rightarrow 0$) by transmission channel impairments.

Error Correction: Detecting the mutation, identifying its precise data coordinates, and flipping it back to its valid native state.

Redundancy Principle: Both mechanisms depend on appending extra, mathematically dependent redundant bits to the raw payload so the receiver can evaluate data integrity.

2. Error Detection Techniques

- **Parity Check:** Appends a single bit ensuring the cumulative count of 1s matches an even/odd constraint. Fails to detect even-numbered bit flips.
- **Checksum:** Segments data into uniform k -bit segments, sums them using 1's complement math, and appends the final inverted total. Heavily deployed within the TCP/IP stack.
- **Cyclic Redundancy Check (CRC):** Applies modulo-2 division on the data block using a specific generator polynomial. The resulting remainder is appended as a robust check field to catch burst errors.

3. Error Correction Techniques

- **Forward Error Correction (FEC):** Fixes errors autonomously without retransmissions. For example, *Hamming Code* maps parity check bits at power-of-2 positions (1, 2, 4, 8 \dots) to compute a binary error pointer (syndrome word).
- **Backward Error Correction (BEC):** Detects errors, silently dumps the broken packet frame, and triggers an *ARQ (Automatic Repeat Request)* notification to command a fresh retransmission.

Quick Summary Blueprint:

- *Error Detection: Asks "Is the data corrupted?" → Low bit overhead → Uses Parity, Checksum, CRC.*
- *Error Correction: Asks "Where is the corruption and how do I fix it?" → High bit overhead → Uses Hamming Code, ARQ.*

| Q21. Explain Store-and-Forward packet switching. (5 Marks)

1. Core Concept

Store-and-Forward packet switching is a fundamental data routing mechanism where an intermediate node (like a network router) must completely ingest, buffer, and check an entire packet before transmitting it toward the next node or destination hop.

2. Operational Steps

1. **Ingress & Buffer:** The router receives incoming streams of bits and caches them inside memory buffers. It cannot begin outbound scheduling until the final bit is stored.
2. **Integrity Audit:** The node runs an algorithmic validation check on the packet header using fields like the FCS/CRC remainder. If corruption is found, the packet is instantly discarded.
3. **Route Selection:** If the packet is clean, the router reads the destination IP header, parses its localized routing tables, maps the best path, and pushes the packet down that line.

3. Key Trade-offs

- **Advantages:** Maximizes network efficiency by dropping malformed packets early, preventing the waste of downstream links. Seamlessly bridges links with mismatched transmission speeds (e.g., buffering a 1 Gbps input down to a 100 Mbps output).
- **Disadvantages:** Accumulates node-by-node serialization delay, making it slower than cut-through alternatives. Demands heavy hardware buffer memory capacity under high congestion loads.

Mathematical Delay Blueprint:

The cumulative transmission delay over N identical links is mapped as:

$$\text{Total Delay} = N \times (L / R)$$

- Where L = packet length (bits) and R = link capacity/bandwidth (bps).

| Q22. Explain design issues of the Network Layer. (5 Marks)

1. Introduction

The core responsibility of the Network Layer is packet transport from a source host to a destination host across heterogeneous networks. To do this efficiently, architects must resolve structural issues at the software and routing interface tiers.

2. Major Design Bottlenecks

- **Transport Layer Decoupling:** The Network Layer must expose uniform, clean interfaces that completely decouple the Transport Layer from hardware dependencies, router layouts, or structural topologies.
- **Subnet Operational Topology:** Architects must choose between a connectionless *Datagram* framework (each packet routes independently with full addresses; can arrive out of order) or a connection-oriented *Virtual Circuit* model (pre-allocated static paths; sequential delivery).
- **Routing Algorithms:** Implementing scalable path-determination logic to resolve the most optimal end-to-end paths while dynamically recovering from link crashes and runtime graph state shifts.
- **Congestion Mitigation:** Policing traffic loads when router buffers fill up, leading to packet loss and performance degradation. Handled via load shedding, choke packets, or traffic policing.
- **Internetworking Protocol Bridging:** Resolving syntax and formatting conflicts when data hops across disparate, sovereign networks with differing addressing modes and Maximum Transmission Units (MTUs) using fragmentation and reassembly.

Design Imperatives:

- *Layer Separation: Keep Transport Layer fully agnostic of the network topology.*
- *Routing & Control: Determine paths while actively preventing buffer overloads.*
- *Interoperability: Map heterogeneous networks cleanly via fragmentation structures.*

| Q23. Explain the TCP Three-Way Handshake mechanism. (5 Marks)

1. Architectural Objective

The TCP Three-Way Handshake is a mechanism designed to establish a reliable, bi-directional virtual channel between a client and a server. It initializes starting **Sequence Numbers** for byte-stream tracking and verifies that both communication hosts have allocated buffer resources and are ready to transmit.

2. Handshake Phase Progression

- **Phase 1 - SYN (Synchronize):** The client transmits an active initialization segment containing a random Initial Sequence Number ($\text{Seq} = x$) with the **SYN** flag set to 1. The client transitions to the **SYN-SENT** state.
- **Phase 2 - SYN-ACK (Synchronize-Acknowledge):** The server intercepts the request, notes the client's tracking state, and responds with its own sync token ($\text{Seq} = y$) while acknowledging the client's data by setting $\text{Ack} = x + 1$. Both the **SYN** and **ACK** flags are active. The server transitions to the **SYN-RCVD** state.
- **Phase 3 - ACK (Acknowledge):** The client processes the server's acknowledgment and dispatches a final verification segment with $\text{Seq} = x + 1$ and $\text{Ack} = y + 1$ (setting the **ACK** flag to 1). Both endpoints transition to the **ESTABLISHED** state, opening the data pipe.

Handshake Blueprint Matrix:

1. Client → Server: SYN ($\text{Seq} = x$)
2. Server → Client: SYN-ACK ($\text{Seq} = y, \text{Ack} = x + 1$)
3. Client → Server: ACK ($\text{Seq} = x + 1, \text{Ack} = y + 1$)